**EEE305 – MTE392**

**Microcontrollers**

**FINAL PROJECT REPORT**

# Automated Water Filtration System


*Submitted by:*

*Group 12*

*Furkan Güntaş*

*Zeynep Ecem Gelmez*

*Emir Cemail*

Project Mentors:

Asst. Prof. Ahmet Fevzi Bozkurt

Faculty of Engineering and Natural Sciences

Kadir Has University


2024 – 2025 Fall

# TABLE OF CONTENT

# 1 INTRODUCTION

**Project Overview**

In this project, a system is built for auto-water filtration for filtering polluted water. The construction contains two essential containers: one on the top holds the unclean water, and the other at the bottom accommodates the filtration unit mechanisms to carry out the filtering job. The system is fitted with a peristaltic pump, which is used to transfer contaminated water from the lower container to the upper filter and return treated water after filtration back to the lower container.

It continues doing this until it reaches complete stability in the cleaning status of the water. If the water has been cleaned, then the outlet of the pump is routed through a servo motor to direct the filtered water into a separate clean container. This creates a separate cycle for both dirty and clean water.

The presented automation system controls the water level using a water level sensor. The sensor continuously monitors the water level inside the lower container and defines the system's operating scenarios. The sensor's readings and the system's status are displayed on an LCD for the user to see.

In addition, the filter system can be controlled and monitored remotely. To this end, a GUI has been designed using MATLAB. This interface gives a view to the user whether the system is functioning or not and allows the running of start/stop instructions or the implementation of the changes.

**Goals of the Project**

The goal of this project can be explained in four main categories. Those will follow up as:

1. Development of Automatic Water Filtration System

- The main objective of the work is to develop a completely automatic water filtration system that does not require manual supervision. It offers an efficient method for the purification of polluted water with the maximum possible production of drinking water. Hence, it aims to save time, energy, and labour compared to traditional methods.

2. Real-Time Monitoring and Control of Filtration Process

The system shall also have sensors and control mechanisms to monitor the filtration process continuously. Components like the water level sensor continuously monitor not only the effectiveness of filtration but also whether the system is functioning or not. In this respect, the filtration process will be carried out with increased reliability and without interruption.

3. Oversight and Regulation of the System via an Interface (MATLAB GUI)

A MATLAB-based GUI has been developed that will enable a user to monitor and operate the system with ease. It will present the status of the system to the user, allow changes in settings, and give full control over the filtration process. Users can operate the system without technical expertise.

4. Oversight of the Automated Operational Cycle Based on Water Level and Filtration Condition

It works in an autonomous working cycle through constant parameter assessment, such as water level and filtration condition. For instance, it switches on the filtration if it detects that the water is dirty until the water becomes clean. If the water is clean, the cycle then stops and moves the water into a clean container. This level of automation enhances energy efficiency and reduces the need for human involvement.

# 2 SYSTEM WORKFLOW

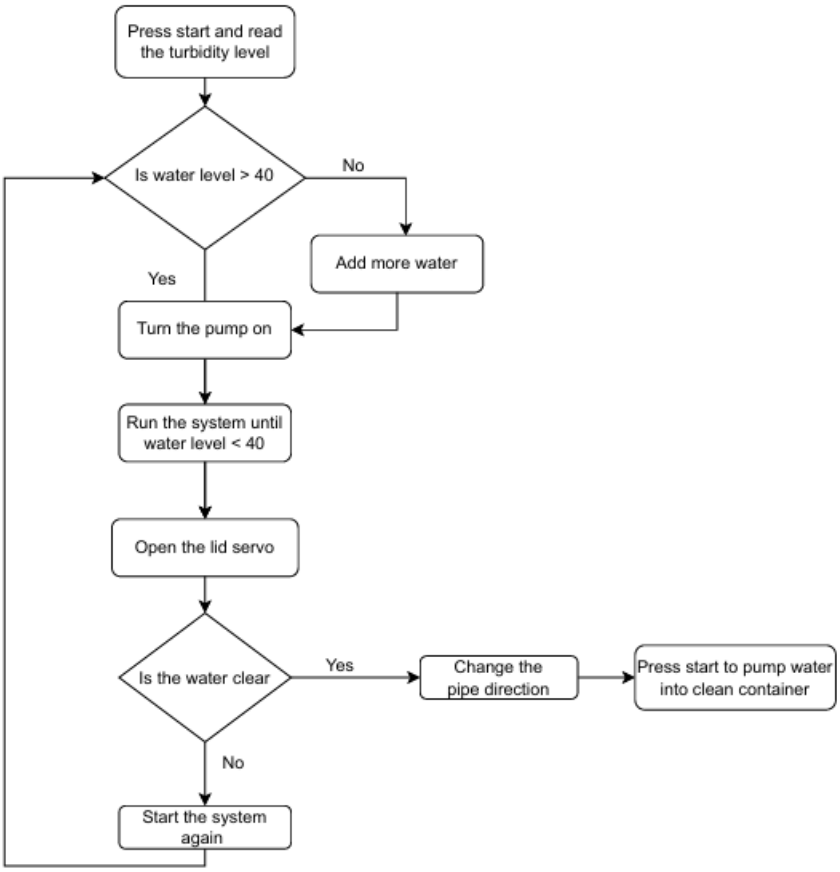


Figure 1. The flowchart of the system shows the workflow.

In figure 1. the flow chart provides a detailed explanation of how an user-controlled water filtration system works. The procedure begins when the user starts the system. The water's turbidity level is measured and evaluated immediately following startup. If the water level exceeds 40 units, the system continues to operate; otherwise, the user is notified that more water must be supplied. This stage guarantees that there is enough water for the system to function normally.

When the water level is appropriate, the pump activates, and the filtration process begins. We chose the threshold of the water level after our test runs. During this procedure, the water is cleaned by transporting it from the bottom container to the higher container with the filter, and the system repeats this until the water level falls below 40 units. Although the filtration cycle appears to be complete at this stage, the procedure is still ongoing. The system opens the lid using a servo motor to inspect and evaluate the cleanliness of the water.

If the water is clean, the servo motor changes the pipe direction and prepares to pump clean water into a separate container. This process is conducted performed using a command initiated by the user. If the water is still unclean, the system returns to the beginning and repeats the filtration process. This cycle continues until the water is entirely clear. The sensor automatically checks water level .

# 3 SYSTEM DESIGN

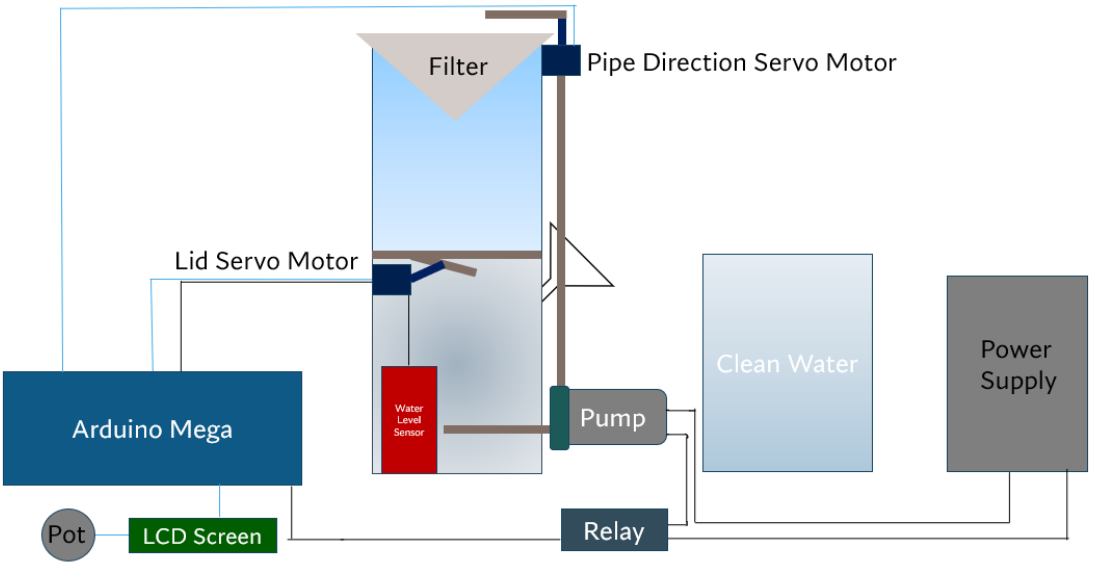


Figure 2. Representation of the system

Figure 2. shows the components of the automatic water filtration system and the relations between them. In its simplest description, the basis of operation is the transportation of the dirty water with the help of a peristaltic pump to the filter, filtration of that water, and distribution of the clean water into a certain container. An Arduino Mega provides automation of this process with a water level sensor, servo motors, a relay module, and an LCD for reading; on top of it all, it uses a MATLAB GUI for user display.

The electronic components we used to create the system are:

Arduino Mega: The microcontroller that controls the system. It processes the data coming from the water level sensor and manages the servo motors and pump.

Figure 3. Peristaltic Dosing Pump.

Peristaltic Pump: It is our mechanical component that transports dirty water to the filtering unit. It is used with relay module to be able control it with Arduino.


Figure 4. The SG90 RC Servo motor.

Servo Motors (2): One is used to direct the water (pipe direction) and the other is used to control the opening and closing of the lid of the container with the filter.

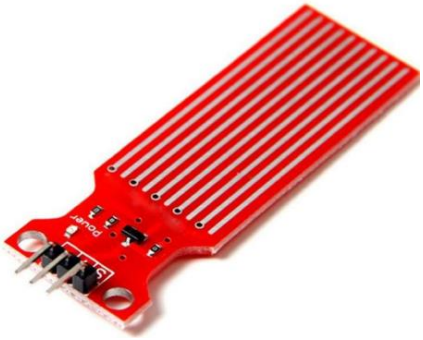

Figure 5. Water level sensor.

Water Level Sensor: It is our sensor that directs the automatic working cycle of the system by measuring the water level.

Relay Module: It allows high-power components such as the pump to be controlled by the Arduino.

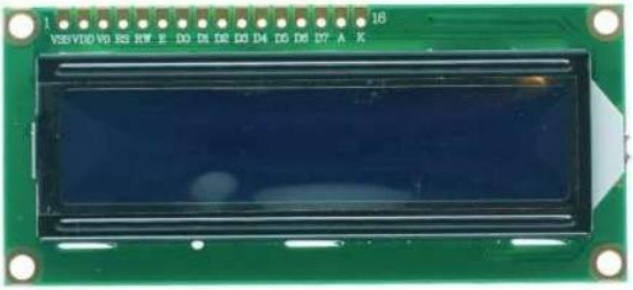

Figure 6. 2x16 LCD Screen.

LCD Screen (16x2): We use it to show the user information such as water level and system status.

Potentiometer: It is used to adjust the contrast of the LCD screen.

# 4 GRAPHICAL USER INTERFACE

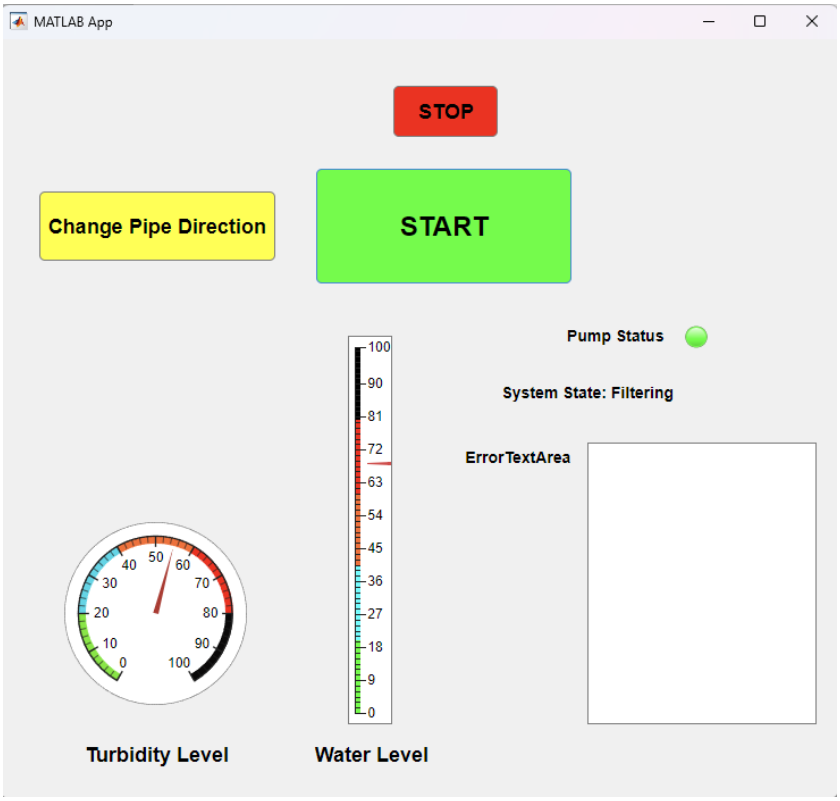


Figure 7. Shows graphical user interface (GUI) designed using MATLAB App Designer.

The figure above is from a GUI developed in MATLAB App Designer; the interface demonstrates all types of information about system status and control to enable easy management by the user. START - This is a green-colored button which basically starts the filtration process and thus gives the command for starting the system. Upon this button being clicked, the system determines the turbidity level of the water and gives an initial value as a turbidity measurement of about 55 units. This initiates the filtration process.

Once the filtration is done, the user can transfer the water to a clean container by using the Change Pipe Direction button in yellow. In this case, the turbidity level is measured again and goes down to 0 units, which means that the water is cleared. Also, the STOP button in red allows for the stopping of the system manually, giving full control to the user in every stage.

The interface also has a set of indicators that support observing the status of both water and the system. The Turbidity Level indicates the turbidity measure of the water by an analogue scale. The Water Level shows the amount of water inside the system using a vertical bar graph. The system also comprises a Pump Status indicator, which is an LED that reflects the state of the pump. The System State field represents the current state of the system in text form, such as "Filtering". The ErrorTextArea is used to log errors and warnings.

This user interface allows for the observation of real-time status and provides intuitive control. Visualization of turbidity and water level enables the user to take further action in managing the filtration process effectively.

# 5 ARDUINO CODE

```
21  void setup() {
22    Serial.begin(38400);
23    pinMode(relayPin, OUTPUT);
24    digitalWrite(relayPin, LOW); // Start with the pump OFF
25
26    // Initialize Servos
27    lidServo.attach(lidServoPin);
28    pipeServo.attach(pipeServoPin);
29    lidServo.write(165);  // Pipe starts in optimal position
30    pipeServo.write(0);  // Pipe starts in optimal position
31
32    // Initialize LCD
33    lcd.begin(16, 2);
34    lcd.print("System Ready");
35    lcd.setCursor(0, 1);
36    lcd.print("Status: Idle");
37  }
```

The setup() function is responsible for initializing all critical components of the water filtration system and ensuring that they are in their default states before the system starts operating. It begins by establishing serial communication at a baud rate of 38400 using Serial.begin(), enabling the Arduino to send and receive data from external devices, such as a MATLAB interface or other controllers. The relay pin, which controls the pump, is configured as an output using pinMode(), and it is set to LOW using digitalWrite(), ensuring the pump remains OFF during initialization to prevent unintended operation.

The servo motors controlling the system's lid and pipe direction are then initialized. The lidServo and pipeServo objects are attached to their respective pins (lidServoPin and pipeServoPin) using the attach() method. Their starting positions are defined using write(), with the lid servo set to position 165, which likely corresponds to a closed or optimal state for the lid, and the pipe servo set to position 0, indicating a neutral or default position for the pipe.

Finally, the LCD display is initialized to provide feedback to the user. The lcd.begin(16, 2) function sets the display to have 16 columns and 2 rows, suitable for displaying system messages. The print() function is used to display "System Ready" on the first row, indicating successful initialization, and "Status: Idle" on the second row, showing that the system is currently inactive and ready for operation. This setup ensures that all components are properly initialized, set to their default safe states, and ready for subsequent user commands or automated processes.

```
void loop() {
  // Read water level
  int waterLevelScale = analogRead(waterLevelPin);
  waterLevel = map(waterLevelScale, 0, 1023, 0, 100);

  // Send water level data to MATLAB GUI
  Serial.print("Water Level:");
  Serial.println(waterLevel);

  if (systemStarted) {
    lcd.setCursor(0, 0);
    lcd.print("Water Level: ");
    lcd.print(waterLevel);
    lcd.print("%  ");

    // Control the lid servo based on water level
    if (waterLevel < 20) {
      lidServo.write(165); // Open the lid
      systemState = "Low Water";
      lcd.setCursor(0, 1);
      lcd.print("Status: Low Water");
      if (pumpRunning) {
        digitalWrite(relayPin, LOW); // Stop the pump
        pumpRunning = false;
      }
    } else if (pumpRunning) {
      lidServo.write(200); // Close the lid
      systemState = "Filtering";
      lcd.setCursor(0, 1);
      lcd.print("Status: Filtering ");
    }
  }
```

This loop() function is the core of the Arduino program, ensuring real-time monitoring, decision-making, and feedback for the water filtration system. The function begins by reading the raw water level data from the sensor connected to waterLevelPin using analogRead. This raw value, which ranges from 0 to 1023, is converted into a percentage (0–100%) using the map function. This percentage value is stored in the variable waterLevel, which represents the current water level in the system.

Next, the function sends the water level data to the MATLAB GUI using the Serial.print and Serial.println functions. The label "Water Level:" is transmitted, followed by the actual percentage value of the water level. This ensures that the GUI remains updated with real-time information from the system, enabling remote monitoring and control.

If the system is active, as indicated by the systemStarted flag, the function updates the LCD to display the current water level. The cursor on the LCD is first set to the beginning of the first row (0, 0) using lcd.setCursor, and then the message "Water Level: " is printed, followed by the water level percentage and the "%" sign. This provides clear, localized feedback to the user about the current water level.

The function then evaluates the water level to determine the appropriate system response. If the water level is below 20%, the lid servo is opened by setting its position to 165 using lidServo.write. The system state is updated to "Low Water", and the LCD displays "Status: Low Water" on the second row. Additionally, if the pump is currently running (pumpRunning is true), the pump is stopped by setting the relayPin to LOW using digitalWrite, and the pumpRunning flag is set to false. This ensures the system prevents further operation of the pump when water levels are critically low, safeguarding the components and preventing damage.

If the water level is 20% or higher and the pump is running, the lid servo is closed by setting its position to 200, indicating normal operation. The system state is updated to "Filtering", and the LCD displays "Status: Filtering" on the second row. This ensures that the system continues its filtering operation when adequate water levels are detected.

```
72  // Listen for GUI commands
73  if (Serial.available() > 0) {
74    String command = Serial.readStringUntil('\n');
75    command.trim();
76
77    if (command == "START") {
78      if (waterLevel >= 30) {
79        systemStarted = true;
80        digitalWrite(relayPin, HIGH); // Start the pump
81        pumpRunning = true;
82        systemState = "Filtering";
83        lcd.setCursor(0, 1);
84        lcd.print("Status: Filtering ");
85        Serial.println("PUMP:ON");
86      } else {
87        Serial.println("ERROR:Low Water");
88      }
89    } else if (command == "STOP") {
90      digitalWrite(relayPin, LOW); // Stop the pump
91      pumpRunning = false;
92      systemState = "Stopped";
93      lcd.setCursor(0, 1);
94      lcd.print("Status: Stopped   ");
95      Serial.println("PUMP:OFF");
96    } else if (command == "CHANGE") {
97      togglePipeServo(); // Change pipe direction
98      Serial.println("PIPE:CHANGED");
99    }
100 }
```

This code snippet is responsible for handling commands sent via serial communication to control the water filtration system, including starting, stopping, and changing the system's pipe direction. It operates continuously within the Arduino's loop() function, ensuring the system is always ready to receive and process user inputs or commands from an external interface such as a MATLAB GUI.

The function begins by checking if there is any data available in the serial buffer using Serial.available(). If data is detected, it reads the incoming command as a string using Serial.readStringUntil('\n'), which captures the message up to a newline character. The command is then cleaned of any extra whitespace using trim(), ensuring only the core instruction is processed.

If the command is "START", the system checks whether the water level is sufficient (≥30%). If so, it activates the system by setting the systemStarted flag to true, turning on the pump using digitalWrite(relayPin, HIGH), and updating the pumpRunning state to true. The system state is set to "Filtering", and this status is displayed on the LCD using lcd.setCursor() and lcd.print(). Simultaneously, a message "PUMP:ON" is sent over the serial interface to notify the external controller that the pump has been successfully started. If the water level is below 40%, the system prevents the pump from starting and sends an error message "ERROR:Low Water" over the serial connection, safeguarding the system from operating under unsafe conditions.

If the command is "STOP", the pump is turned off using digitalWrite(relayPin, LOW), and the pumpRunning state is updated to false. The system state is changed to "Stopped", and this new status is displayed on the LCD. Additionally, a message "PUMP:OFF" is sent via the serial connection to inform the external controller that the pump has been stopped successfully.

If the command is "CHANGE", the function calls the togglePipeServo() function to change the direction of the pipe, allowing the system to redirect the flow of water as needed. A confirmation message "PIPE:CHANGED" is then sent via serial communication, indicating that the pipe direction has been successfully updated.

# 6 MATLAB GUI CODE

```
properties (Access = private)
        arduinoObj                % Arduino connection object
        timer                     % Timer object for real-time updates
    end

    methods (Access = private)

        % Timer callback for real-time updates
function TimerCallback(app)
    try
        % Check if data is available from Arduino
        if app.arduinoObj.NumBytesAvailable > 0
            data = readline(app.arduinoObj); % Read data from Arduino

            % Parse water level data
            if startsWith(data, "Water Level:")
                waterLevel = str2double(extractAfter(data, ":"));
                app.WaterLevelGauge.Value = waterLevel; % Update water level gauge

                % Update system state and pump status
                if waterLevel < 40
                    % Low water level actions
                    app.SystemStateLabel.Text = "System State: Low Water";
                    app.PumpStatusLamp.Color = [1 0 0]; % Red
                    app.ErrorTextArea.Value = "Warning: Low water level!";
                else
                    % Sufficient water level actions
                    app.SystemStateLabel.Text = "System State: Filtering";
                    app.PumpStatusLamp.Color = [0 1 0]; % Green
                    app.ErrorTextArea.Value = ""; % Clear warnings
                end
```

The MATLAB TimerCallback function is a critical component of the water filtration system, enabling real-time monitoring and updating the graphical user interface (GUI) based on the data received from the Arduino. The function begins by checking whether there is any data available from the Arduino using NumBytesAvailable. If data is present, it reads a line of data using the readline function, ensuring the function processes only new and relevant information.

Once data is received, the function determines its type by checking if it starts with "Water Level:" using the startsWith function. If the condition is met, the numeric water level value is extracted from the string using extractAfter and converted to a numerical format using str2double. This extracted water level value is then assigned to the WaterLevelGauge in the GUI, visually updating the water level in percentage form for the user.

The function then evaluates the water level to decide the appropriate system state and updates the GUI accordingly. If the water level is below 40%, the system enters a "Low Water" state. In this state, the SystemStateLabel is updated to display "System State: Low Water", the PumpStatusLamp is turned red ([1 0 0]), and a warning message, "Warning: Low water level!", is displayed in the ErrorTextArea. These actions ensure the user is immediately alerted to the critical condition, preventing further operation that could harm the system.

If the water level is 40% or higher, the system is set to normal operation, labeled as "Filtering". The SystemStateLabel is updated to "System State: Filtering", the PumpStatusLamp is turned green ([0 1 0]), and any previous warnings in the ErrorTextArea

are cleared. This ensures that the user sees a clear indication that the system is functioning correctly under adequate conditions.

This function not only monitors the water level continuously but also ensures that the user interface reflects the current state of the system in real-time. By dynamically adjusting the pump status lamp, water level gauge, and system state labels, the function provides an intuitive and informative interface for the user. Additionally, the function includes a safety mechanism by preventing operation when water levels are critically low, ensuring reliable and safe system behavior.

```
        % Button pushed function: STARTButton
        function STARTButtonPushed(app, event)
    try
        % Check if arduinoObj exists and reconnect if necessary
        if isempty(app.arduinoObj) || ~isvalid(app.arduinoObj)
            app.arduinoObj = serialport("COM4", 38400);
            configureTerminator(app.arduinoObj, "LF");
            flush(app.arduinoObj);
        end

        % Send START command to Arduino
        writeline(app.arduinoObj, "START");

        % Check if the timer exists and restart it
        if isempty(app.timer) || strcmp(app.timer.Running, "off")
            app.timer = timer('ExecutionMode', 'fixedRate', ...
                              'Period', 0.1, ...
                              'TimerFcn', @(~, ~) TimerCallback(app));
            start(app.timer);
        end

        % Update GUI
        app.PumpStatusLamp.Color = [0 1 0]; % Green for active pump
        app.SystemStateLabel.Text = "System State: Filtering";
        app.ErrorTextArea.Value = "Pump started successfully.";
        app.TurbidityLevelGauge.Value = 55;
    catch
        % Display error if unable to start
        app.ErrorTextArea.Value = "Error: Unable to start the pump.";
        app.PumpStatusLamp.Color = [1 0 0]; % Red for error
        app.SystemStateLabel.Text = "System State: Stopped";
    end
```

The STARTButtonPushed function is executed when the "START" button in the GUI is clicked, and it is responsible for initializing communication with the Arduino, starting the filtration system, setting up real-time updates via a timer, and updating the GUI to reflect the system's operational state. We used a try-catch function block to prevent the application from crashing in case of communication or startup errors and to ensure robust error handling.

The function first checks whether the Arduino connection object (app.arduinoObj) exists and is valid. If the connection is missing or invalid, a new serial connection is established to the Arduino on port "COM4" with a baud rate of 38400. The configureTerminator function is used to set the serial communication terminator to "LF" (Line Feed), which helps in properly identifying the end of messages from the Arduino. The communication buffer is then flushed using flush(app.arduinoObj) to clear any residual or old data, ensuring the connection is ready for fresh communication.

After establishing the connection, the "START" command is sent to the Arduino via the writeline function. This command activates the water filtration system on the Arduino side, signaling it to begin its operation.

Next, the function ensures that a timer object (app.timer) exists and is running. The timer is used for periodic real-time updates in the GUI. If the timer object is uninitialized (isempty(app.timer)) or not running (strcmp(app.timer.Running, "off")), a new timer is created. This timer is configured with a fixed-rate execution mode ('fixedRate') and a period of 0.1 seconds, meaning it triggers every 100 milliseconds. The timer's callback function is set to TimerCallback, which is responsible for updating the GUI with the latest system data. The timer is then started using the start function.

Once the Arduino and timer are set up, the GUI is updated to reflect the system's active state. The pump status lamp (app.PumpStatusLamp) is set to green ([0 1 0]), indicating that the pump is running. The system state label (app.SystemStateLabel) is updated to display "System State: Filtering", showing that the system is filtering water. Additionally, the error text area (app.ErrorTextArea) is updated with the message "Pump started successfully." to confirm to the user that the system is operational. A default value of 55 is set for the turbidity level gauge (app.TurbidityLevelGauge), which might represent an initial or safe operating level.

If any part of the process fails—such as issues with the Arduino connection, sending the command, or starting the timer—the catch block is executed. Within this block, the error text area is updated with the message "Error: Unable to start the pump." to notify the user of the issue. The pump status lamp is set to red ([1 0 0]), signaling an error state, and the system state label is updated to "System State: Stopped" to reflect that the system is inactive.

# 7 CONCLUSION

The Automated Water Filtration System developed in this project successfully addresses the critical need for efficient and autonomous water purification. The integration of hardware components like the Arduino Mega, peristaltic pump, water level sensor, and servo motors with a MATLAB-based GUI enabled the creation of a fully functional and user-friendly system.
Key accomplishments of the project include:

1. Automation and Efficiency: The system operates independently to filter polluted water, reducing the need for manual intervention and ensuring energy-efficient operation.
2. Real-Time Monitoring: The use of sensors and a well-designed graphical interface allows users to monitor the filtration process and system status seamlessly.
3. User Accessibility: The MATLAB GUI simplifies system control, making the technology accessible to users without technical expertise.
4. Modularity and Expandability: The design is modular, allowing for future enhancements such as additional sensors or more sophisticated algorithms to improve performance further.

The system effectively showcases how combining automation and user-friendly interfaces can provide sustainable and scalable solutions to pressing dirty water filtration.